APP DEVELOPMENT PROJECT REPORT

Trekking Management Application

1. Student Details

Name: Arnav Raj

Roll Number: 23f3002537

Email: 23f3002537@gmail.com

About Me: I am a student pursuing BS in Data Science along with B.tech in Computer Science. I have interest in Software development, Machine learning and data-driven technologies. I enjoy learning new concepts and developing practical applications.

2. Project Details

Project Title: Trekking Management Application

Problem Statement

To develop a centralized trekking management system where administrators can manage treks and staff, users can browse and book available treks, and trek staff can manage their assigned treks and participants.

Approach

The application was developed using Flask as the backend web framework with a modular project structure. It supports three roles: Admin, Trek Staff, and User.

- **Admin**: Manages treks, users, staff, trek assignments, bookings, and search operations.
- **Trek Staff**: Views assigned treks and participants registered for those treks.
- **User/Trekker**: Browses available treks, filters treks, views trek details, books treks, views bookings and history, and manages a profile.

The application follows role-based access control. Flask Blueprints separate Authentication, Admin, Staff, and User routes.

3. AI/LLM Declaration

I used ChatGPT (GPT-5) to assist with Flask route structuring, SQLAlchemy model relationships, Jinja2 template organization, debugging, and documentation formatting. The approximate extent of AI/LLM usage in the project is **35%**.
All AI-assisted suggestions were reviewed, modified, integrated, and tested manually during application development.

4. Technologies and Frameworks Used

Technology / Library	Purpose / Role in Project
Python	Primary backend programming language
Flask	Backend framework, routing, and request handling
Flask-SQLAlchemy	ORM and database operations
SQLite	Relational database and application persistence
Jinja2	Dynamic server-side HTML rendering
Bootstrap 5	Responsive UI, forms, cards, tables, and navigation
Flask-Login	Authentication and session management
Werkzeug	Password hashing and password verification
python-dotenv	Loading environment variables from .env
HTML / CSS	Page structure and additional styling
GitHub	Remote repository and project backup

5. Database Schema / ER Diagram

The application database contains four main tables: users, treks, bookings, and trek_assignments.

1. **Users Table**

The users table stores Admin, Staff, and regular User accounts. The role field differentiates application roles, while the status field supports approval and blacklist-related account control.

- id
- full_name
- email
- password_hash
- phone
- role
- status
- created_at
- updated_at

2. Treks Table

The treks table stores trekking activity details, capacity, dates, description, and status.

- id
- name
- location
- difficulty
- duration
- total_slots
- available_slots
- description
- start_date

- end_date
- status
- created_at
- updated_at

3. Bookings Table

The bookings table stores trek bookings made by users. user_id references users.id and trek_id references treks.id. A composite unique constraint on user_id and trek_id prevents duplicate booking records for the same user and trek.

- id
- user_id
- trek_id
- booking_date
- status

4. Trek Assignments Table

The trek_assignments table manages staff-to-trek assignments. staff_id references users.id and trek_id references treks.id. Staff members are stored in the users table with role='staff'.

A composite unique constraint on staff_id and trek_id prevents duplicate assignment records.

- id
- trek_id
- staff_id
- assigned_at

Database Relationships

Relationship	Cardinality	Description
User → Booking	1 : N	One user can have multiple bookings
Trek → Booking	1 : N	One trek can have multiple bookings
User (Staff) → TrekAssignment	1 : N	One staff member can have multiple assignment records
Trek → TrekAssignment	1 : N	One trek can have multiple assignment records in the current schema

The Booking table acts as an associative entity between Users and Treks and also stores booking date and booking status.

Foreign key relationships:

- bookings.user_id → users.id
- bookings.trek_id → treks.id
- trek_assignments.staff_id → users.id
- trek_assignments.trek_id → treks.id

ER Diagram:


users
id integer
full_name varchar(100) NN
email varchar(120) NN
password_hash varchar(255) NN
phone varchar(15) NN
role varchar(10) NN
status varchar(20) NN
created_at datetime NN
updated_at datetime NN
treks
id integer
name varchar(100) NN
location varchar(100) NN
difficulty varchar(20) NN
duration integer NN
total_slots integer NN
available_slots integer NN
description text
start_date date NN
end_date date NN
status varchar(20) NN
created_at datetime NN
updated_at datetime NN
bookings
id integer
user_id integer NN
trek_id integer NN
booking_date datetime NN
status varchar(20) NN
trek_assignments
id integer
trek_id integer NN
staff_id integer NN
assigned_at datetime NN
dbdiagram.io

6. API Resource Endpoints

Authentication Routes:

Endpoint	Method	Description
/auth/login	GET, POST	Display login page and authenticate user
/auth/register	GET, POST	Register a new user
/auth/logout	GET	Logout the current user

Admin Routes :

Endpoint	Method	Description
/admin/dashboard	GET	Display administrator dashboard
/admin/treks	GET	Display all treks
/admin/treks/add	GET, POST	Create a new trek
/admin/treks/<trek_id>/edit	GET, POST	Edit trek details
/admin/treks/<trek_id>/delete	POST	Delete a trek
/admin/staff	GET	Display staff members
/admin/staff/<staff_id>/approve	POST	Approve staff member
/admin/staff/<staff_id>/blacklist	POST	Blacklist staff member
/admin/assign-staff	GET, POST	Assign staff to a trek
/admin/users	GET	Display registered users
/admin/bookings	GET	Display all bookings
/admin/search	GET, POST	Search users, staff, and treks

User Routes :

Endpoint	Method	Description
/user/dashboard	GET	Display user dashboard
/user/treks	GET	Browse available treks
/user/treks/<trek_id>	GET	Display trek details
/user/treks/<trek_id>/book	GET/POST	Book selected trek
/user/bookings	GET	Display user's bookings
/user/history	GET	Display trekking history
/user/profile	GET, POST	View and update user profile

Staff Routes:

Endpoint	Method	Description
/staff/dashboard	GET	Display staff dashboard
/staff/treks	GET	Display assigned treks
/staff/treks/<trek_id>	GET	Manage assigned trek
/staff/treks/<trek_id>/participants	GET	Display trek participants
/staff/participants	GET	Display participants from

		assigned treks
/staff/profile	GET, POST	View and update staff profile

API Definition

The project primarily follows a server-rendered Flask architecture. The api.yaml file documents the application's HTTP routes and resources in OpenAPI format. The core application renders Jinja2 templates and does not claim to be a separate JSON REST API.

7. Architecture and Features

Architecture Overview:

app.py
The main application entry point initializes Flask, loads configuration, initializes extensions, registers Blueprints, configures Flask-Login, and creates database tables programmatically.

config.py
The configuration file manages application settings. Environment variables are loaded from the .env file so sensitive configuration values are not directly hardcoded.

extensions.py
This file contains reusable Flask extension objects such as SQLAlchemy and Flask-Login. Separating extension initialization helps reduce circular import problems.

models/
The models directory contains SQLAlchemy models: user.py, trek.py, booking.py, and trek_assignment.py. Each model defines database columns, relationships, and constraints. models/__init__.py exports the model classes.

routes/
The routes directory contains auth.py, admin.py, staff.py, and user.py. Flask Blueprints separate functionality according to authentication and application roles.

templates/
Jinja2 templates are separated into admin, auth, staff, and user directories. Role-specific base templates maintain consistent sidebar navigation. Child templates extend admin/base.html, staff/base.html, or user/base.html as required.

static/
The static directory stores CSS. Bootstrap provides the primary responsive UI layout.

utils/
The utilities directory contains reusable functionality. decorators.py defines admin_required, staff_required, and user_required role-based decorators.

Implemented Features:

Authentication and Authorization

- User registration, login, and logout
- Secure password hashing and verification
- Flask session management
- Role-based redirection after login
- Protected routes using login_required

Admin Features

- Dashboard statistics and recent bookings
- Add, edit, delete, and view treks
- View users and staff members
- Approve and blacklist staff
- Assign staff members to treks
- View all bookings
- Search users, staff, and treks

User Features

- User dashboard and available trek browsing
- Filter treks by difficulty and location
- View detailed trek information
- Book available treks
- Prevent duplicate trek bookings
- View personal bookings and trekking history

Staff Features

- Staff dashboard
- View assigned treks
- View participant statistics
- Manage assigned treks
- View participants for assigned treks
- View and update staff profile

Security Features

Password Hashing

Passwords are not stored as plain text. Werkzeug generate_password_hash() and check_password_hash() are used through the User model's set_password() and check_password() methods.

Environment Variables

Sensitive configuration is stored in the .env file and loaded using python-dotenv. The .env file is excluded from Git using .gitignore.

8. Video Presentation

Drive Link: https://drive.google.com/file/d/1gJHzataNVJTFLiwXkW40XgWS7Ne0IvW0/view?usp=sharing